AJAY.I - 71812401007 · MODULE 2 — AGENT DESIGN AND REASONING NOTES

1. System 1 and System 2 Reasoning

1.1 What is System 1 and System 2 Reasoning?

Simple meaning:

System 1 and System 2 are two ways of thinking.

- System 1 is fast thinking. It makes decisions quickly without spending much time analyzing.
- System 2 is slow thinking. It carefully analyzes a problem before making a decision.

For example, when you see `2 + 2`, you immediately know the answer is 4. This is System 1.

When you solve a difficult mathematical problem, you think step by step. This is System 2.

Technical explanation:

System 1 and System 2 are two reasoning modes described in dual-process theory.

System 1 uses fast, automatic and intuitive processing, whereas System 2 uses deliberate, analytical and computationally intensive reasoning.

In Agentic AI, these concepts help explain how an agent can perform simple tasks quickly and use more reasoning for complex tasks.

1.2 System 1 Reasoning

Simple meaning: The agent makes a quick decision using a familiar pattern.

Technical explanation: System 1 reasoning performs rapid, automatic inference with relatively low computational cost.

It is useful when the task is simple, familiar or requires an immediate response.

Examples:

- Identifying a common object.
- Answering a simple factual question.
- Detecting a familiar pattern.
- Selecting a predefined action.

Example:

Input — "What is 5 + 5?" → **System 1** — Recognizes a simple calculation → **Output: 10**

1.3 System 2 Reasoning

Simple meaning: The agent carefully thinks about the problem before answering.

Technical explanation: System 2 reasoning uses deliberate analysis, multiple reasoning steps, planning and evaluation to solve complex problems.

It is useful when:

- The problem has multiple steps.
- The answer requires calculations.
- Several possible actions exist.
- The agent must evaluate risks.
- A mistake could produce an incorrect result.

Example:

User asks:

"Find the average of five students' marks, identify the highest mark and calculate the percentage."

The agent must:

1. Identify the marks.
2. Calculate the total.
3. Calculate the average.
4. Find the highest mark.
5. Calculate the percentage.
6. Present the results.

This requires more deliberate reasoning.

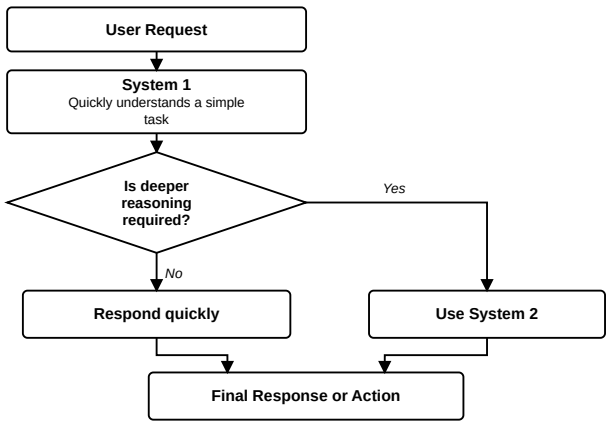
1.4 System 1 vs System 2

System 1	System 2
Fast reasoning	Slow, deliberate reasoning
Automatic processing	Analytical processing
Low computational cost	Higher computational cost
Suitable for simple tasks	Suitable for complex tasks
Usually fewer reasoning steps	Usually multiple reasoning steps
Example: Recognizing a cat	**Example:** Planning a robot's route

1.5 How AI agents use both

A modern AI agent can use both reasoning modes.

DIAGRAM



How an agent uses System 1 and System 2

Example: An agent can answer "What is 2 + 2?" immediately, but use detailed planning to organize a three-day travel itinerary.

2. Test-Time Compute

2.1 What is Test-Time Compute?

Simple meaning:

Test-Time Compute is nothing but giving an AI model more time or computational resources to think about a question before answering.

For example, instead of immediately answering a difficult mathematics question, the model can spend more time solving it step by step.

Technical explanation:

Test-Time Compute refers to the computational resources used by an AI model during inference to improve the quality of its output.

It may involve generating additional reasoning steps, exploring multiple candidate solutions, performing verification or using additional model calls before producing the final response.

Unlike training, test-time compute happens when the model is already being used.

2.2 How it works

User Question → **AI Model** — Analyze the question → **Allocate More Compute** — Reason, explore and verify → **Final Answer** — Return a more carefully evaluated result

2.3 Techniques used in Test-Time Compute

1. Extended reasoning

The model generates more reasoning steps before answering.

Example: Solving a complex mathematical problem.

2. Multiple candidate solutions

The model generates different possible answers and compares them.

Example: An agent explores several routes before choosing the safest one.

3. Verification

The model checks whether its answer is correct.

Example: A coding agent runs its generated program and checks the output.

4. Tool-assisted computation

The model uses external tools to perform calculations, execute code or retrieve information.

Example: An agent uses Python to calculate a large dataset's average.

2.4 Advantages

- Can improve reasoning accuracy.
- Helps solve complex problems.
- Supports verification.
- Enables exploration of alternative solutions.

2.5 Limitations

- Requires additional computational resources.

- Can increase response time.
- May increase inference cost.
- More reasoning does not guarantee a correct answer.

Important: Test-Time Compute improves the opportunity for better reasoning; it does not guarantee correctness.

3. Self-Correction Mechanisms

3.1 What is Self-Correction?

Simple meaning:

Self-correction is nothing but an AI agent checking its own work, finding mistakes and trying to fix them.

For example, an AI agent writes Python code, runs it, finds an error and changes the code.

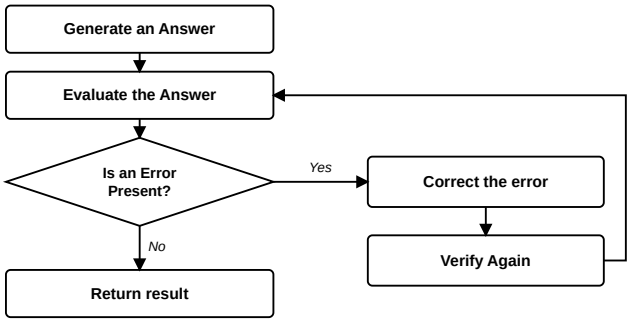
Technical explanation:

Self-correction is a mechanism through which an AI agent evaluates its generated output or actions, identifies errors and performs corrective steps using feedback, verification or additional reasoning.

It helps an agent improve the quality of its output without requiring the user to identify every mistake.

3.2 Working of Self-Correction

DIAGRAM



Working of self-correction

3.3 Types of Self-Correction

1. Reflection-Based Correction

Simple meaning: The agent looks at its previous answer and thinks about what went wrong.

Technical explanation: The agent evaluates its previous reasoning or output and generates an improved result based on identified weaknesses.

Example: An agent notices that its answer did not include all requested points and revises it.

2. Execution-Based Correction

Simple meaning: The agent runs its work and checks whether it actually works.

Technical explanation: The agent uses execution feedback, such as program output, exceptions or test results, to identify and correct errors.

Example: A coding agent runs Python code and fixes a `NameError`.

3. Feedback-Based Correction

Simple meaning: The agent uses feedback from the environment or another system to improve its next action.

Technical explanation: The agent incorporates external feedback, evaluation results or reward signals into its decision-making process.

Example: A robot changes its route after receiving information that the previous route is blocked.

4. Verification-Based Correction

Simple meaning: The agent checks whether the final answer satisfies the required conditions.

Technical explanation: The agent compares its output against predefined constraints, expected results or validation criteria.

Example: Checking whether a generated JSON response contains all required fields.

3.4 Python Example

```python
# Self-correction example
answer = 10 * 5
expected = 50

if answer == expected:
    print("Answer is correct")
else:
    answer = expected
    print("Corrected answer:", answer)
```

Output:

```
Answer is correct
```

This example demonstrates basic verification. A real AI self-correction mechanism additionally needs a way to generate or revise the answer.

3.5 Advantages

- Reduces avoidable errors.
- Improves output quality.
- Helps coding agents debug programs.
- Supports reliable tool execution.
- Enables feedback-based improvement.

Limitation: An agent can incorrectly judge its own output as correct. Independent testing or external verification is often necessary.

4. Comparative Analysis of Agent Frameworks

Your syllabus includes three important approaches:

- AutoGen
- CrewAI
- CoALA

First understand what an agent framework is.

4.1 What is an Agent Framework?

Simple meaning:

An agent framework is nothing but a collection of tools and structures that makes it easier to build AI agents.

Instead of writing everything from the beginning, developers use a framework to organize the agent's tasks, tools and interactions.

Technical explanation:

An agent framework provides abstractions and components for building, coordinating and executing AI agents.

It may support tool integration, task planning, memory, communication, execution and multi-agent coordination.

4.2 AutoGen

Simple meaning:

AutoGen helps developers build AI systems in which multiple agents or AI components communicate and work together.

Technical explanation:

AutoGen is an agent framework associated with Microsoft that supports the development of conversational and multi-agent applications.

It provides mechanisms for agent interaction, tool use and coordinated task execution.

Example

A software development system can contain:

- Coding agent.
- Testing agent.
- Reviewing agent.

The coding agent generates code.

The testing agent checks it.

The reviewing agent evaluates the result.

These agents communicate to complete the task.

Main features

- Agent communication.
- Multi-agent workflows.
- Tool integration.
- Task coordination.
- Human involvement in workflows.

4.3 CrewAI

Simple meaning:

CrewAI is nothing but a framework that allows multiple AI agents to work as a team, with each agent having a specific role.

For example, one agent researches information, another writes a report and another checks the report.

Technical explanation:

CrewAI is a framework for building role-based multi-agent systems in which agents are assigned roles, goals and tasks and can collaborate through structured

workflows.

Example

Suppose you want to create a research report.

Agent	Role
Researcher	Collects information
Writer	Creates the report
Reviewer	Checks the report

The agents work together to complete the task.

Main features

- Role-based agents.
- Task assignment.
- Agent collaboration.
- Workflow coordination.
- Tool integration.

4.4 CoALA

Simple meaning:

CoALA helps developers understand how a language agent should organize its memory, thinking and actions.

Technical explanation:

CoALA, or Cognitive Architectures for Language Agents, is a conceptual framework for designing language agents using cognitive architecture principles.

It emphasizes the interaction between memory, reasoning, action and learning.

Unlike AutoGen and CrewAI, CoALA is primarily an architectural framework rather than a specific multi-agent orchestration library.

Main components

- Memory.
- Internal reasoning.
- Action.
- Learning.

4.5 Comparison Table

Feature	AutoGen	CrewAI	CoALA
Main purpose	Agent interaction and coordination	Role-based agent collaboration	Language-agent architecture
Approach	Conversational workflows	Roles and tasks	Memory, reasoning and action
Agent structure	Agents communicate	Agents have assigned roles	Cognitive components
Multi-agent support	Yes	Yes	Can inform multi-agent design
Tool usage	Supported	Supported	Architectural concept
Example	Agents discussing a solution	Researcher, writer, reviewer	Agent memory and reasoning design

Exam point: AutoGen emphasizes agent interaction, CrewAI emphasizes role-based collaboration, and CoALA emphasizes the cognitive organization of language agents.

5. Agent Environments

5.1 What is an Agent Environment?

Simple meaning:

An agent environment is nothing but the place where an AI agent performs its tasks.

For example, a robot's environment is the physical world around it.

A game-playing agent's environment is the game.

A web agent's environment is the website it interacts with.

Technical explanation:

An agent environment is the external system or world in which an agent receives observations, executes actions and obtains feedback.

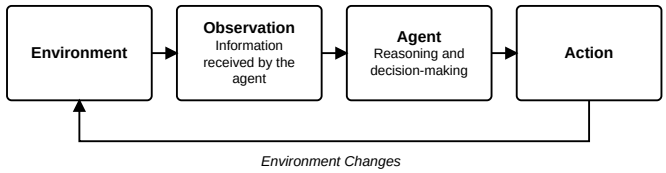
The environment defines the available actions, observations, rules and conditions under which the agent operates.

5.2 Components of an Agent Environment

Component	Meaning
Agent	The system that makes decisions
Environment	The world in which the agent operates
Observation	Information received by the agent
Action	What the agent performs
State	Current condition of the environment
Reward	Feedback about the agent's performance

5.3 Agent Environment Diagram

DIAGRAM



Agent environment diagram

5.4 Types of Agent Environments

1. Fully Observable Environment

Simple meaning: The agent can see all the information required to understand the current state.

Technical explanation: A fully observable environment provides the agent with sufficient information to determine the relevant current state.

Example: A chess-playing agent that can see the entire chessboard.

2. Partially Observable Environment

Simple meaning: The agent cannot see everything happening in the environment.

Technical explanation: A partially observable environment provides incomplete or uncertain observations of the underlying state.

Example: A robot that cannot see objects behind a wall.

3. Deterministic Environment

Simple meaning: The same action in the same state produces the same result.

Example: Moving a chess piece according to the rules.

4. Stochastic Environment

Simple meaning: The same action may produce different results.

Example: A robot moving on a slippery surface.

5. Static Environment

Simple meaning: The environment does not change while the agent is thinking.

Example: Solving a fixed mathematics problem.

6. Dynamic Environment

Simple meaning: The environment can change while the agent is thinking.

Example: A self-driving car operating in traffic.

6. Gymnasium Framework

6.1 What is Gymnasium?

Simple meaning:

Gymnasium is nothing but a Python library that provides environments where AI agents can practice making decisions.

For example, you can create a game environment and train an agent to choose the correct actions.

Technical explanation:

Gymnasium is an open-source Python framework for developing and evaluating reinforcement learning agents.

It provides standardized environment interfaces for interacting with simulations through observations, actions, rewards and episode termination.

It is the maintained successor to OpenAI Gym.

6.2 Why Gymnasium is used

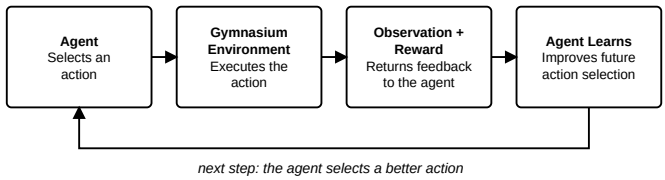
Gymnasium allows developers to:

- Create simulated environments.
- Define possible actions.
- Receive observations.

- Calculate rewards.
- Train reinforcement learning agents.
- Evaluate agent performance.

6.3 Gymnasium Working

DIAGRAM



Gymnasium working

6.4 Important Gymnasium Functions

Function	Purpose
`gym.make()`	Creates an environment
`env.reset()`	Starts or resets the environment
`env.step()`	Executes an action
`env.render()`	Displays the environment when supported
`env.close()`	Closes the environment

6.5 Python Example

Install Gymnasium:

```
pip install gymnasium
```

Create a simple environment:

```
import gymnasium as gym

env = gym.make("CartPole-v1")
observation, info = env.reset()
print("Initial observation:", observation)

action = env.action_space.sample()
observation, reward, terminated, truncated, info =
env.step(action)

print("Action:", action)
print("Reward:", reward)
env.close()
```

Explanation

- `gym.make()` creates the CartPole environment.
- `reset()` starts a new episode.
- `action_space.sample()` selects a random available action.
- `step()` executes the action.
- `reward` provides feedback.
- `terminated` indicates whether the task's terminal condition was reached.
- `truncated` indicates whether the episode ended due to an external limit, such as a time limit.

Important: This example performs a random action. It does not train an intelligent agent.

7. POMDP Environments

7.1 What is POMDP?

Simple meaning:

POMDP is nothing but a decision-making environment where the AI agent cannot see the complete situation.

It must make decisions using incomplete information.

For example, a robot cannot see what is behind a wall, but it still needs to decide where to move.

Technical explanation:

POMDP stands for Partially Observable Markov Decision Process.

It is a mathematical framework used to model decision-making in environments where the agent receives incomplete or uncertain observations of the underlying state.

The agent uses observations, actions, rewards and a belief about the hidden state to make decisions.

7.2 Components of POMDP

A POMDP is commonly represented using:

POMDP = (S, A, T, O, Z, R, γ)

Symbol	Meaning
S	Set of environment states
A	Set of possible actions
T	State transition model
O	Set of observations
Z	Observation model
R	Reward function
γ	Discount factor

1. State

The actual condition of the environment.

Example: The robot's actual location.

2. Action

The action performed by the agent.

Example: Move left.

3. Transition Model

Simple meaning: Explains how the environment changes after an action.

Technical explanation: The transition model describes the probability of reaching a new state after taking an action in the current state.

4. Observation

The information received by the agent.

Example: A camera detects an object.

5. Observation Model

Explains how likely an observation is to occur when the environment is in a particular state.

6. Reward

Provides feedback about the quality of the agent's action.

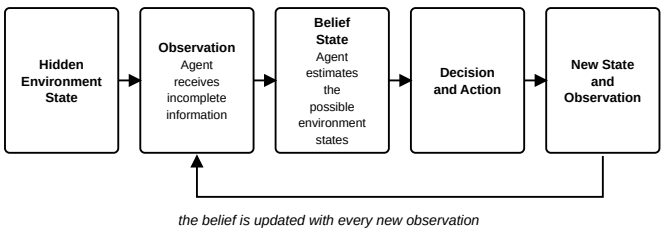
Example: The robot receives a positive reward for reaching its destination.

7. Discount Factor

Controls the importance of future rewards compared with immediate rewards.

7.3 POMDP Working

DIAGRAM



POMDP working

7.4 Belief State

Simple meaning:

A belief state is nothing but the agent's current estimate of what might be happening.

For example, if a robot cannot see an object, it may estimate that the object is behind the wall.

Technical explanation:

A belief state is a probability distribution over possible environment states.

It allows the agent to reason about uncertainty when the actual state is not directly observable.

7.5 Example

A robot searches for a person in a building.

The robot cannot see the person directly.

It receives information from cameras and sensors.

Based on these observations, it estimates the person's possible location.

The robot chooses a search action and updates its belief using new observations.

This is a POMDP-style decision process.

7.6 MDP vs POMDP

MDP	POMDP
Markov Decision Process	Partially Observable Markov Decision Process
State is available to the agent	State is not fully observable
Uses state for decision-making	Uses observations and belief states
Example: Fully visible board game	**Example:** Robot searching behind obstacles

Exam point: POMDP is useful for robotics, autonomous navigation, uncertain environments and decision-making with incomplete information.

8. OSWorld

8.1 What is OSWorld?

Simple meaning:

OSWorld is nothing but a simulated computer environment where AI agents can practice using an operating system.

For example, an agent can open applications, interact with windows, edit files and perform computer tasks.

Technical explanation:

OSWorld is a benchmark and environment for evaluating computer-use agents on real-world operating system tasks.

It provides tasks involving desktop applications, operating system interfaces and computer interaction.

It helps evaluate whether an AI agent can perform tasks through graphical interfaces rather than only answering text questions.

8.2 Examples of OSWorld Tasks

- Open a document.
- Edit a file.
- Use a spreadsheet.
- Interact with desktop applications.
- Navigate graphical interfaces.
- Perform operating system tasks.

8.3 Working

User Task — Edit a document → **AI Agent** — Plans the required operations → **Computer Interaction** — Clicks, typing and application operations → **Task Evaluation** — Checks whether the requested task was completed

8.4 Importance

OSWorld helps evaluate:

- Computer-use capabilities.
- GUI interaction.
- Task completion.
- Planning.
- Visual understanding.
- Agent reliability.

Important: OSWorld is primarily an evaluation benchmark and environment, not a general-purpose AI agent framework.

9. WebArena

9.1 What is WebArena?

Simple meaning:

WebArena is nothing but a simulated website environment where AI agents practice completing tasks using a browser.

For example, an agent can search for products, update information or interact with website features.

Technical explanation:

WebArena is a benchmark for evaluating autonomous web agents in realistic, interactive web environments.

It provides website-based tasks that require agents to navigate webpages, understand interface elements and perform actions.

9.2 Examples

An agent may be asked to:

- Search for a product.
- Find information on a website.
- Navigate multiple webpages.
- Update an account setting.
- Complete a web-based workflow.

9.3 Working

User Request — Find a product and add it to a cart → **Agent Planning** — Identify the required browser actions → **Browser Interaction** — Search, click and navigate → **Task Evaluation** — Check whether the task was completed

9.4 OSWorld vs WebArena

OSWorld	WebArena
Focuses on operating system and desktop tasks	Focuses on web-based tasks
Uses desktop applications and interfaces	Uses websites and browser interfaces
Example: Edit a document	**Example:** Search a website
Evaluates computer-use agents	Evaluates web agents

Exam point: OSWorld evaluates operating-system interaction, whereas WebArena evaluates web-based interaction.

10. Vision-Language Models (VLMs)

10.1 What is a Vision-Language Model?

Simple meaning:

A Vision-Language Model is nothing but an AI model that can understand both images and text.

For example, you can give it a photograph and ask:

"What is present in this image?"

The model analyzes the image and answers using language.

Technical explanation:

A Vision-Language Model (VLM) is an AI model that processes visual information and language together.

It can connect image features with textual representations to perform tasks such as image understanding, visual question answering, object recognition and image-based reasoning.

10.2 Why VLMs are important in Agentic AI

Traditional language models mainly process text.

But many agent environments contain visual information.

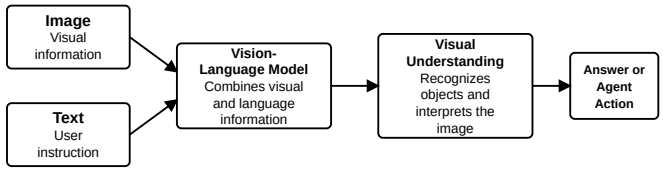
Examples:

- A robot uses a camera.
- A computer-use agent sees screenshots.
- A web agent examines webpage layouts.
- A manufacturing agent analyzes machine images.

VLMs help agents understand these visual inputs.

10.3 Working of a VLM

DIAGRAM



Working of a Vision-Language Model

10.4 Applications

1. Robotics

A robot uses a camera to identify objects and decide where to move.

2. Healthcare

A VLM can assist with interpreting medical images, subject to appropriate clinical validation.

3. Computer-use agents

An agent analyzes screenshots to identify buttons, menus and interface elements.

4. Manufacturing

A system analyzes images to detect defects in products.

5. Autonomous vehicles

A vehicle uses visual information to identify roads, signs and objects.

10.5 Example

User provides an image of a traffic signal and asks:

"What is the signal showing?"

The VLM:

1. Processes the image.
2. Identifies the traffic signal.
3. Recognizes the displayed color.
4. Connects the visual information to the question.
5. Produces a text response.

Important: VLMs can misinterpret images. Critical decisions require validation.

11. Agent Perception

11.1 What is Agent Perception?

Simple meaning:

Agent perception is nothing but how an AI agent understands information from its surroundings.

For example, a robot uses a camera to see an obstacle.

Technical explanation:

Agent perception is the process of collecting, processing and interpreting environmental observations to construct information useful for decision-making.

It converts raw input into meaningful representations of the environment.

11.2 Types of Perception

1. Visual Perception

Simple meaning: Understanding images and videos.

Examples:

- Object recognition.
- Image classification.
- Scene understanding.

2. Audio Perception

Simple meaning: Understanding sound.

Examples:

- Speech recognition.
- Sound detection.
- Voice commands.

3. Text Perception

Simple meaning: Understanding written information.

Examples:

- User instructions.
- Documents.
- Website text.

4. Sensor Perception

Simple meaning: Understanding information collected from sensors.

Examples:

- Temperature.
- Distance.
- Motion.
- GPS readings.

11.3 Perception Process

Raw Input — Images, text, audio or sensors → **Data Processing** — Clean and interpret the input → **Feature / Information Extraction** — Identify useful information → **Environment Understanding** — Produce information for reasoning

11.4 Example: Robot Perception

A robot is moving in a room.

1. The camera captures an image.
2. The perception system identifies a chair.
3. The system estimates the chair's position.
4. The agent recognizes that the chair is an obstacle.
5. The decision-making component selects a suitable route.
6. The robot moves around the chair.

Exam point: Perception provides environmental information, while reasoning uses that information to select actions.

12. Agentic Design Patterns

12.1 What is an Agentic Design Pattern?

Simple meaning:

An agentic design pattern is nothing but a common method used to organize how an AI agent solves a task.

For example, an agent can solve a problem by planning steps first, checking its work or repeatedly observing and acting.

Technical explanation:

An agentic design pattern is a reusable architectural approach for structuring an agent's reasoning, task execution, tool interaction and feedback mechanisms.

Your syllabus includes four important patterns:

1. ReAct.
2. Reflection.
3. Planning.
4. Multi-Agent Systems.

13. ReAct Pattern

13.1 What is ReAct?

Simple meaning:

ReAct is nothing but an AI agent that thinks, performs an action, observes the result and then thinks again.

For example, an agent searching for information on the internet:

- Thinks about what to search.
- Searches the web.
- Reads the result.
- Decides what to do next.

Technical explanation:

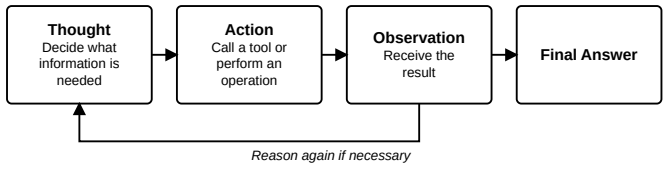
ReAct stands for Reasoning and Acting.

It is an agent design pattern that combines reasoning with external actions.

The agent alternates between reasoning, action execution and observation to solve a task.

13.2 ReAct Working

DIAGRAM



ReAct working

13.3 Example

User asks:

"Find the capital of France."

The agent:

Thought: I need geographical information.

Action: Search for the capital of France.

Observation: The search result identifies Paris.

Thought: The information is sufficient.

Final answer: Paris is the capital of France.

13.4 Advantages

- Combines reasoning and action.
- Supports tool usage.
- Allows decisions to change based on observations.
- Useful for dynamic environments.
- Supports multi-step tasks.

Important: ReAct is not just a sequence of fixed actions. The agent can use the observation from one action to decide what to do next.

14. Reflection Pattern

14.1 What is Reflection?

Simple meaning:

Reflection is nothing but an AI agent looking back at its previous work and identifying what could be improved.

For example, an agent writes a report, checks whether it contains all the required information and then improves it.

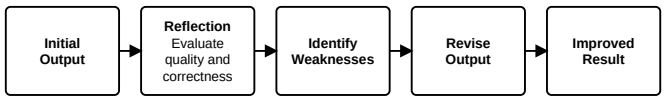
Technical explanation:

Reflection is an agent design pattern in which an agent evaluates previous reasoning, actions or outputs and uses the evaluation to improve subsequent behavior.

It may involve critique, error identification, revision and verification.

14.2 Working

DIAGRAM



Reflection working

14.3 Example

An agent generates Python code.

Initial output contains an incorrect variable name.

The reflection component checks the code and identifies the mistake.

The agent corrects the variable name and generates a revised version.

14.4 Advantages

- Improves output quality.
- Helps identify mistakes.
- Supports iterative improvement.
- Useful for writing and coding agents.

Difference from ReAct: ReAct focuses on reasoning and acting through observations. Reflection focuses on evaluating and improving previous work.

15. Planning Pattern

15.1 What is Planning?

Simple meaning:

Planning is nothing but deciding the steps required to achieve a goal before performing them.

For example, if an agent must book a ticket, it needs to decide which steps to perform and in what order.

Technical explanation:

Planning is an agent design pattern in which an agent decomposes a goal into smaller tasks and determines an action sequence that can achieve the goal.

The plan may be revised when the environment changes or an action fails.

15.2 Types of Planning

1. Forward Planning

Simple meaning: Start from the current situation and determine the steps needed to reach the goal.

Technical explanation: Forward planning searches from the initial state toward a goal state.

Example:

Current location → Route → Destination.

2. Backward Planning

Simple meaning: Start from the goal and determine what must happen before reaching it.

Technical explanation: Backward planning works from the desired goal state toward the conditions required to achieve it.

Example:

Goal: Submit a report.

Determine:

- Report must be completed.
- Data must be collected.
- Data must be analyzed.
- Research must be performed.

3. Hierarchical Planning

Simple meaning: Break a large task into smaller tasks.

Technical explanation: Hierarchical planning decomposes a high-level goal into subtasks and lower-level actions.

Example:

Goal: Organize a trip.

- Select destination.
- Book transport.
- Book accommodation.
- Prepare itinerary.

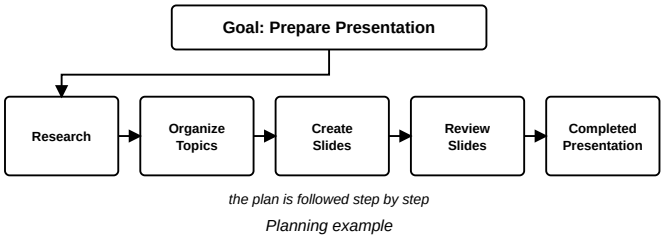
15.3 Planning Example

User asks:

"Prepare a presentation about Agentic AI."

The agent creates a plan:

DIAGRAM



the plan is followed step by step

Planning example

15.4 Advantages

- Breaks complex tasks into manageable steps.
- Supports goal-oriented behavior.
- Helps organize tool calls.
- Improves task execution.
- Allows the agent to track progress.

16. Multi-Agent Systems

16.1 What is a Multi-Agent System?

Simple meaning:

A Multi-Agent System is nothing but a group of AI agents working together to complete a task.

For example, one AI agent collects information, another analyzes it and another prepares the final report.

Technical explanation:

A Multi-Agent System (MAS) consists of multiple autonomous agents that interact, communicate, coordinate or collaborate within a shared environment to achieve individual or common objectives.

Each agent may have its own role, knowledge, tools and decision-making process.

16.2 Why Multi-Agent Systems are used

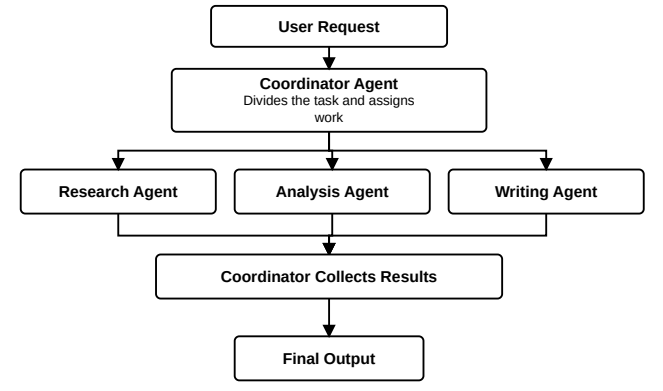
A single agent may find it difficult to perform a large task.

A multi-agent system divides the task among specialized agents.

This allows different agents to focus on different responsibilities.

16.3 Multi-Agent Architecture

DIAGRAM



Multi-agent architecture

16.4 Types of Multi-Agent Systems

1. Cooperative Multi-Agent System

Simple meaning: Agents work together toward a common goal.

Technical explanation: Cooperative agents coordinate their actions and share information to achieve a shared objective.

Example: Multiple robots working together to move a heavy object.

2. Competitive Multi-Agent System

Simple meaning: Agents compete against one another.

Technical explanation: Competitive agents pursue objectives that may conflict with the objectives of other agents.

Example: Two AI agents competing in a chess game.

3. Hybrid Multi-Agent System

Simple meaning: Agents cooperate in some tasks and compete in others.

Technical explanation: Hybrid systems combine cooperative and competitive interactions.

Example: Agents collaborate within teams while competing against other teams.

16.5 Multi-Agent Communication

Agents can communicate through:

- Messages.
- Shared memory.
- Task assignments.
- Tool outputs.
- Common environments.

Communication helps agents exchange information and coordinate decisions.

16.6 Advantages

- Task specialization.
- Parallel task execution.
- Collaboration.
- Modular design.
- Improved handling of complex tasks.

16.7 Limitations

- Communication overhead.
- Coordination complexity.
- Conflicting decisions.
- Increased computational cost.
- Difficulty managing errors across agents.

Exam point: Multi-agent systems use multiple autonomous agents to solve complex tasks through cooperation, coordination or competition.

17. Comparison of Agentic Design Patterns

This is an important comparison because all four patterns solve different problems.

Pattern	Main purpose	Example
ReAct	Reason, act and observe	Search the web and use results
Reflection	Evaluate and improve previous output	Correct generated code
Planning	Organize steps toward a goal	Plan a trip
Multi-Agent	Divide work among agents	Researcher and writer agents

How to remember the difference

- ReAct: Think and act.
- Reflection: Check and improve.
- Planning: Decide steps.
- Multi-Agent: Work together.

18. Important 2-Mark Questions

These answers are written in simple exam-ready language.

1. What is System 1 reasoning?

System 1 reasoning is fast, automatic and intuitive thinking used for simple or familiar tasks with relatively low computational cost.

2. What is System 2 reasoning?

System 2 reasoning is deliberate and analytical thinking used to solve complex problems through multiple reasoning steps.

3. Define Test-Time Compute.

Test-Time Compute refers to the computational resources used during inference to improve an AI model's reasoning, verification or output quality.

4. What is self-correction?

Self-correction is the process by which an AI agent evaluates its output, identifies errors and generates an improved result using feedback or verification.

5. What is AutoGen?

AutoGen is an agent framework associated with Microsoft that supports conversational agent interaction, tool usage and multi-agent workflows.

6. What is CrewAI?

CrewAI is a framework for building role-based AI agents that collaborate through assigned tasks and workflows.

7. Expand CoALA.

CoALA stands for Cognitive Architectures for Language Agents.

8. What is an agent environment?

An agent environment is the external world or system in which an AI agent receives observations, performs actions and obtains feedback.

9. What is Gymnasium?

Gymnasium is an open-source Python framework that provides standardized environments for developing and evaluating reinforcement learning agents.

10. Expand POMDP.

POMDP stands for Partially Observable Markov Decision Process.

11. What is a belief state?

A belief state is a probability distribution over possible environment states that an agent uses when the actual state is not fully observable.

12. What is OSWorld?

OSWorld is a benchmark and environment for evaluating AI agents on operating system and desktop computer-use tasks.

13. What is WebArena?

WebArena is a benchmark for evaluating AI agents on interactive web-based tasks involving browser navigation and website operations.

14. Define Vision-Language Model.

A Vision-Language Model is an AI model that processes visual information and language together to perform image understanding and language-based tasks.

15. What is agent perception?

Agent perception is the process of collecting, processing and interpreting environmental information to support decision-making.

16. What is ReAct?

ReAct is an agent design pattern that combines reasoning and action by allowing an agent to perform actions, observe results and reason again.

17. What is reflection in Agentic AI?

Reflection is the process of evaluating previous reasoning, actions or outputs to identify weaknesses and improve subsequent results.

18. What is planning in Agentic AI?

Planning is the process of decomposing a goal into smaller tasks and determining an action sequence to achieve it.

19. What is a Multi-Agent System?

A Multi-Agent System consists of multiple autonomous agents that interact, communicate or coordinate to achieve individual or shared objectives.

20. Differentiate OSWorld and WebArena.

OSWorld evaluates agents on operating system and desktop tasks, whereas WebArena evaluates agents on interactive web-based tasks.

19. Important 10-Mark Questions

Prepare these questions using the concepts and diagrams above.

1. Explain System 1 and System 2 reasoning with suitable examples and compare their characteristics.
2. Explain Test-Time Compute and self-correction mechanisms in Agentic AI.
3. Compare AutoGen, CrewAI and CoALA based on architecture, purpose and agent coordination.
4. Explain agent environments and their classifications with suitable examples.
5. Explain the Gymnasium framework, its working and basic Python implementation.
6. Explain POMDP environments, their components, belief states and working with a suitable example.
7. Explain OSWorld and WebArena and compare their applications in evaluating AI agents.
8. Explain Vision-Language Models and agent perception with architecture and applications.
9. Explain the ReAct, Reflection and Planning design patterns with diagrams and examples.
10. Explain Multi-Agent Systems, their classifications, architecture, communication, advantages and limitations.

20. Diagram Practice

For your staff's instruction to prepare diagrams, these are the main diagrams to practice from Module 2.

System 1 / System 2

Fast reasoning vs deliberate reasoning

Self-Correction

Generate → Evaluate → Correct

Gymnasium

Agent → Environment → Reward

POMDP

Observation → Belief → Action

ReAct

Thought → Action → Observation

Multi-Agent

Coordinator → Specialized Agents